ShortP2P Protocol

Language: English

Author: Igor Evdokimov

Repository: https://github.com/IskraMmessenger/ShortP2P

Created: 2026-11-11

Modified: 2026-11-11

Related: docs/encryption, docs/messages, protocol.md

Contents

Title page

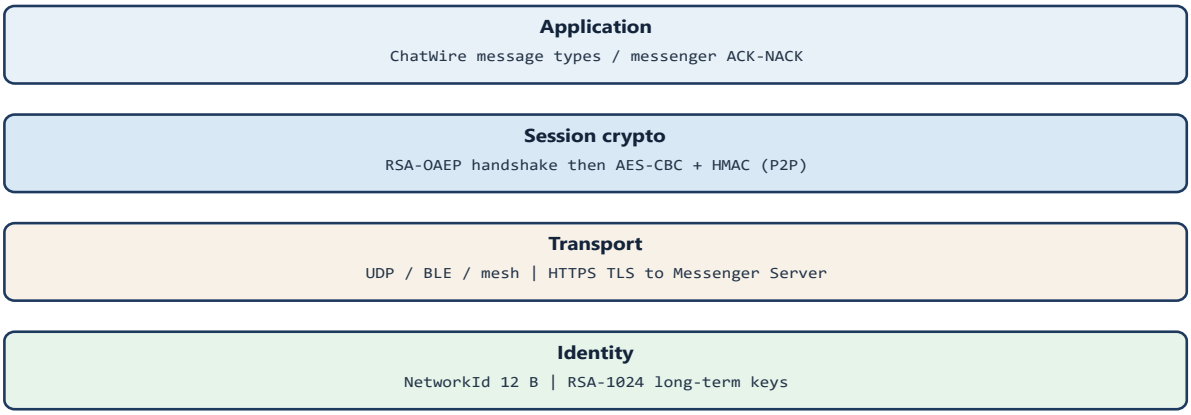
Contents

Shared diagrams

Protocol description

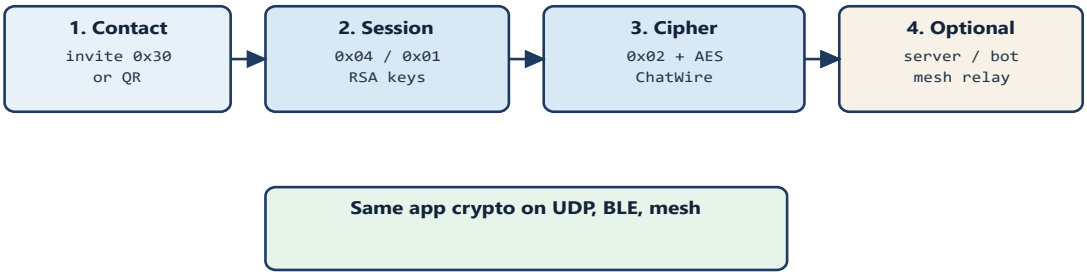
Shared diagrams

Protocol layers

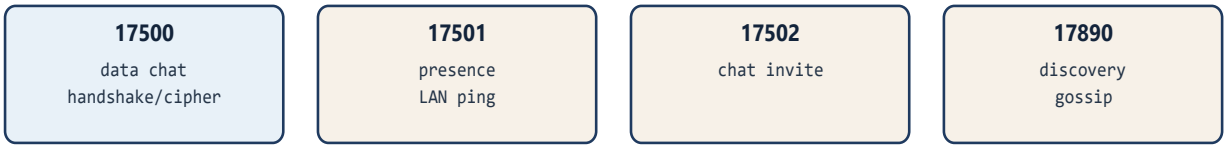


Logical stack (not OS layers). Details: docs/encryption, docs/messages.

Typical direct chat flow



Default UDP ports



Bluetooth and mesh reuse logical frames; HTTPS used for Messenger Server.

Protocol description

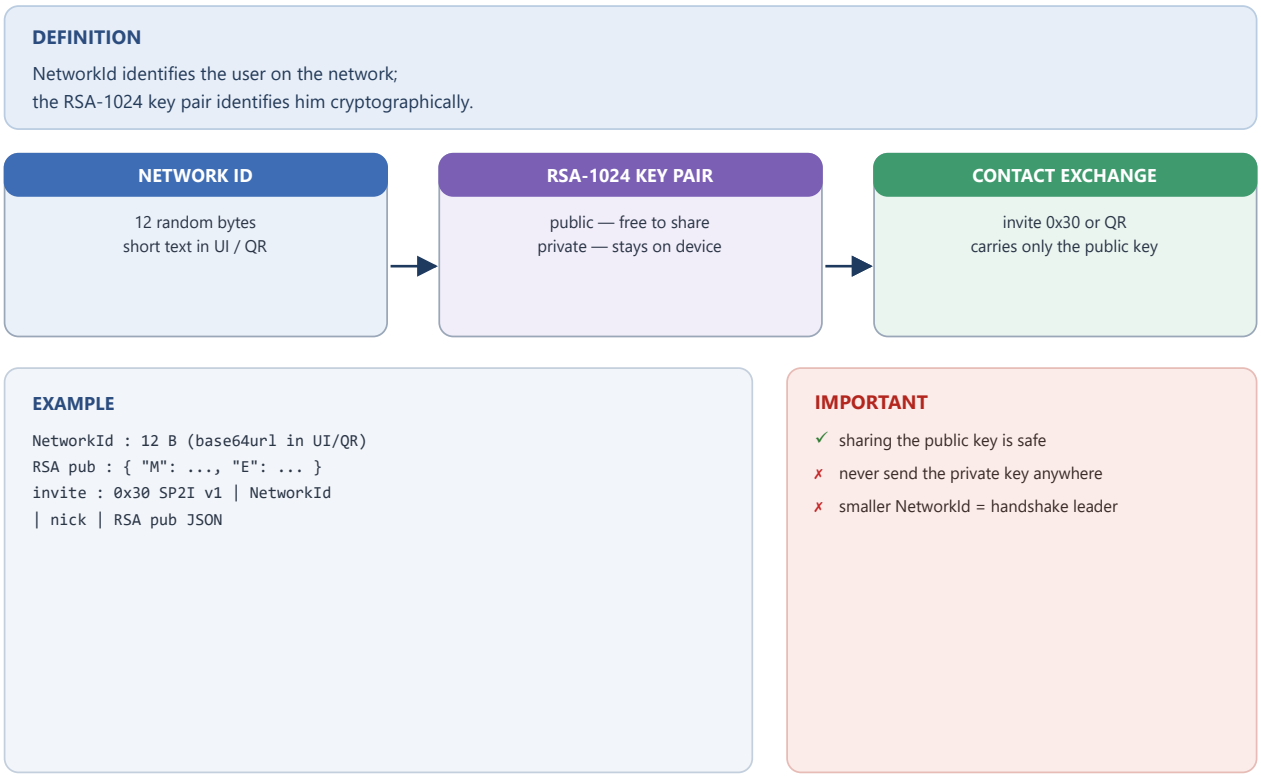
ShortP2P is a peer-to-peer messaging stack. In plain words: two devices (peers) find each other, agree on secret keys, and then exchange chat messages that nobody in between can read. Transport can be direct (UDP, Bluetooth, mesh/LoRa radio) or indirect, through an optional Messenger Server that only relays already-encrypted data.

BASIC TERMS (no prior cryptography knowledge required)

- Peer: the other device / person you chat with.
- NetworkId: 12 random bytes that identify a user, similar to a network address. In the UI and QR codes it is shown as a short base64url text (~16 characters).
- RSA-1024: an asymmetric algorithm — a pair of keys. The public key is shared openly (it travels inside invites and QR codes); the private key never leaves the device. Data encrypted with the public key can be decrypted only with the matching private key. In ShortP2P it is used once per session, to deliver the session secrets to the peer.
- Session keys: two random byte strings created for one chat session: an AES key (16 bytes) and a MAC key (32 bytes). After the handshake both sides hold the same two strings.
- AES-128-CBC: a symmetric cipher — the same key encrypts and decrypts. CBC mode chains blocks so identical plaintext pieces look different; PKCS7 padding aligns the length to a multiple of 16 bytes.
- IV (initialization vector): 16 fresh random bytes generated for every packet, so the same text sent twice produces different ciphertext.
- HMAC-SHA256 tag: a keyed checksum (16 bytes are used) computed over IV + ciphertext. The receiver recomputes it and compares in constant time; a mismatch means the packet was tampered with or corrupted, and it is rejected.
- ChatWire: ShortP2P's own binary format for chat content. It is what gets encrypted and sent; the receiver decrypts it first, then parses it.
- S2P1: a 4-byte "magic signature" (ASCII characters "S2P1") at the start of every ChatWire frame. It works like a file header — the first thing the parser checks to recognize the format.
- Kind byte: the byte right after the magic. Its value says which message type follows.
- ACK / NACK: internal delivery reports. ACK = "message fully received"; NACK = "please resend the chunks with these numbers".
- TLS/HTTPS: the standard transport encryption between a client and the Messenger Server (the padlock icon in browsers).
- JWT: a signed token issued by the server at login. The client attaches it to every API request; the server verifies the signature instead of asking for the password again.
- End-to-end (E2E): only the two chatting devices can read the content. The Messenger Server stores and relays it as an opaque encrypted blob (Base64 text) and cannot read it.
- Bot: a program account on the server that clients can talk to; it also receives only encrypted content.
- Gossip / discovery: auxiliary traffic answering "who is nearby and how to reach them" (UDP port 17890).

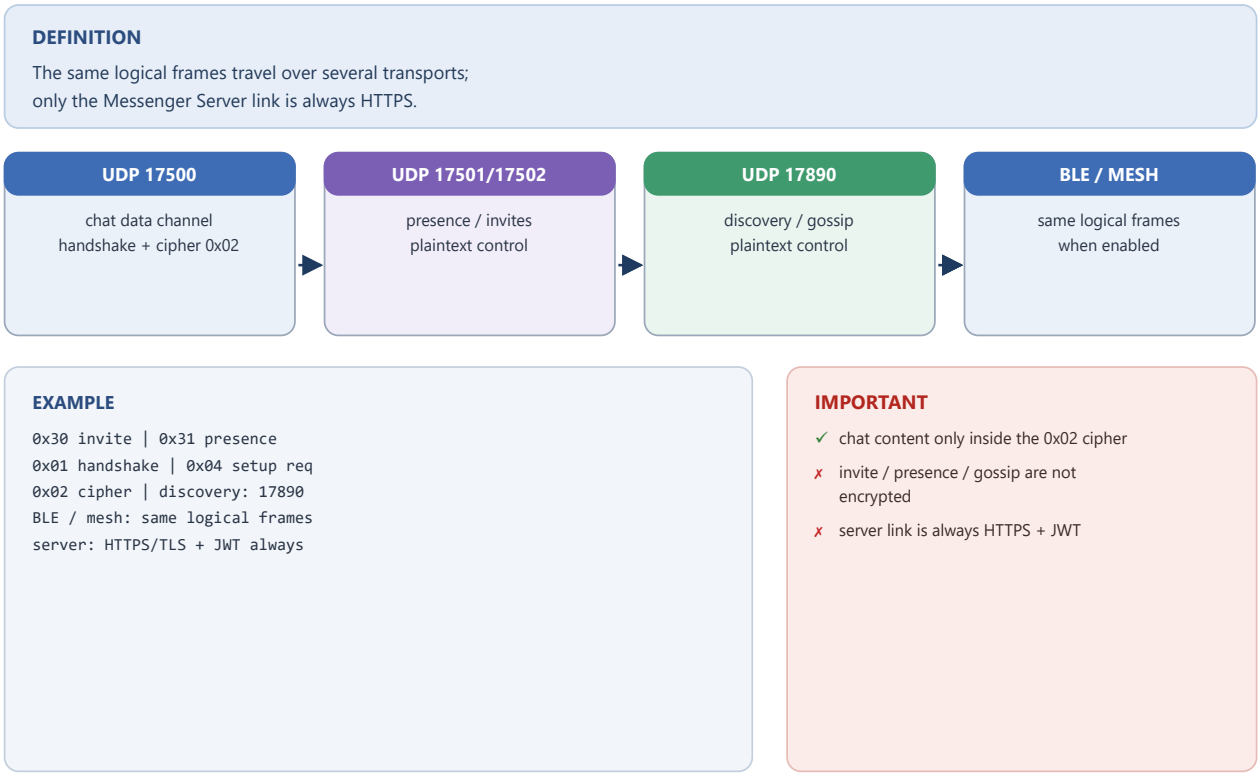
- RELAY: a LAN routing frame that can carry an already-encrypted payload across intermediate nodes.

1) IDENTITY



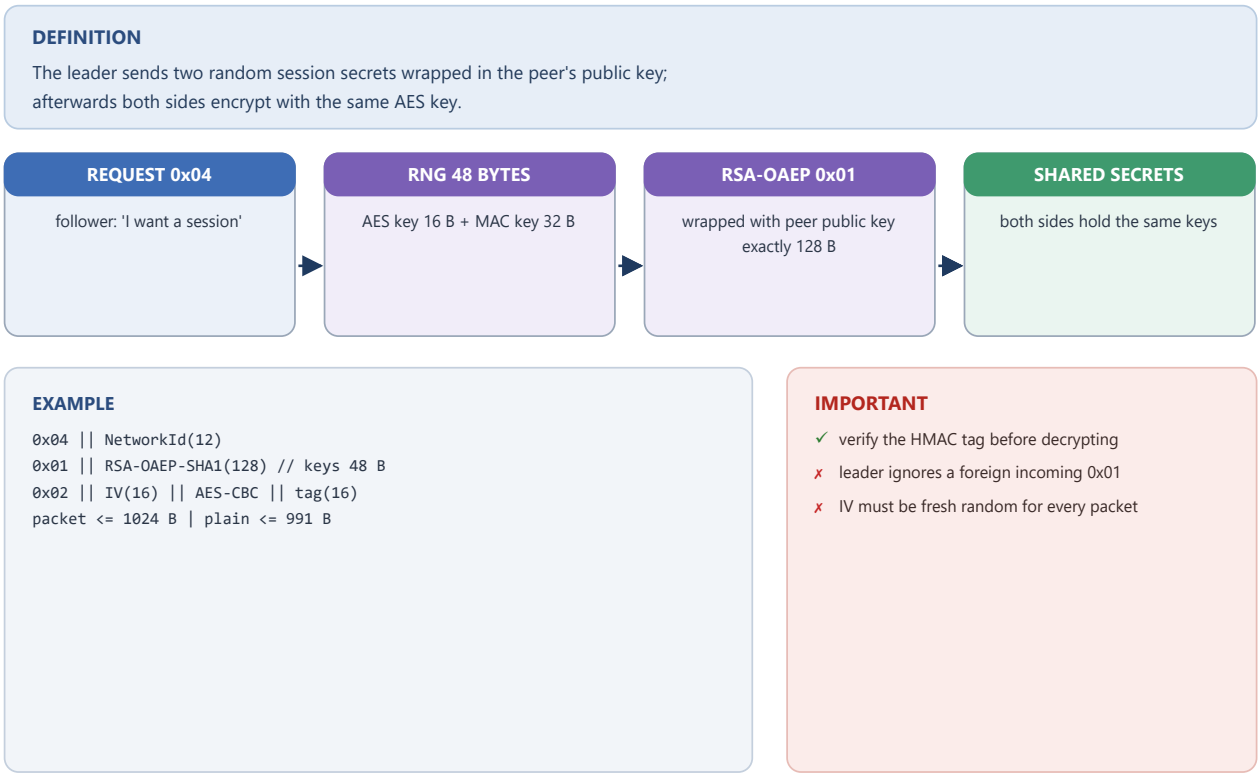
Every user has a NetworkId (12 random bytes) and a long-term RSA-1024 key pair generated at registration. The public key travels in plaintext inside chat invites (0x30) or QR codes — this is safe, a public key is meant to be shared. Rule: the peer with the smaller NetworkId becomes the "leader" of the crypto handshake; the other side waits.

2) TRANSPORTS



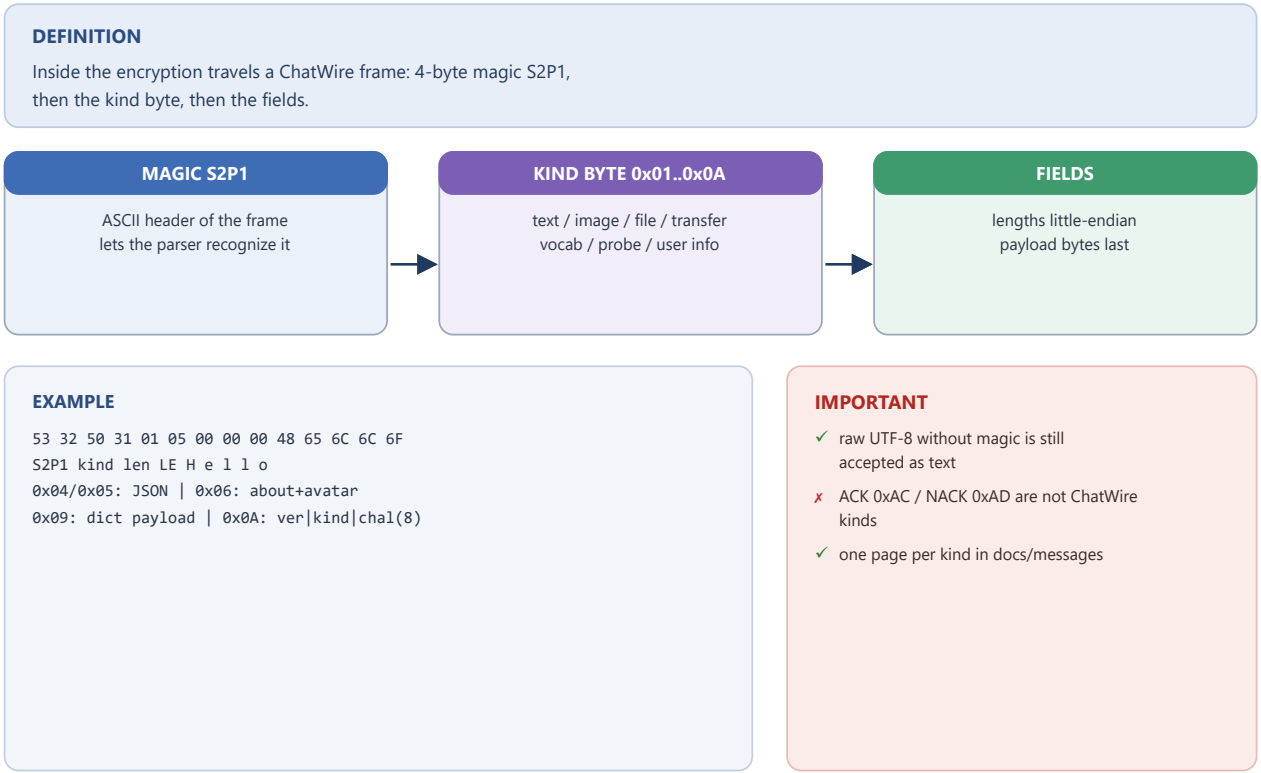
UDP ports by default: 17500 — chat data channel (handshake and encrypted packets), 17501 — presence ("I am online") announcements, 17502 — chat invites, 17890 — discovery/gossip.
Bluetooth and mesh/LoRa transports carry the same logical frames when enabled.
Communication with the Messenger Server always goes over HTTPS/TLS with JWT authentication.

3) SESSION (direct P2P), step by step



a. The follower may send frame 0x04 with its NetworkId: "I want a session".
b. The leader generates two random secrets (AES key 16 B + MAC key 32 B) and encrypts this 48-byte bundle with the peer's RSA public key using RSA-OAEP(SHA-1). The result is exactly 128 bytes and is sent as frame 0x01.
c. The follower decrypts it with its RSA private key; now both sides share the same two secrets.
d. Every following data frame is 0x02 + packet, where packet = IV(16) + AES-CBC ciphertext + HMAC tag(16), at most 1024 bytes. The receiver verifies the tag first, then decrypts.

4) APPLICATION MESSAGES (inside the encryption)



The plaintext fed into the cipher is usually a ChatWire frame: magic "S2P1", then the kind byte, then the fields (lengths are little-endian unless noted otherwise).

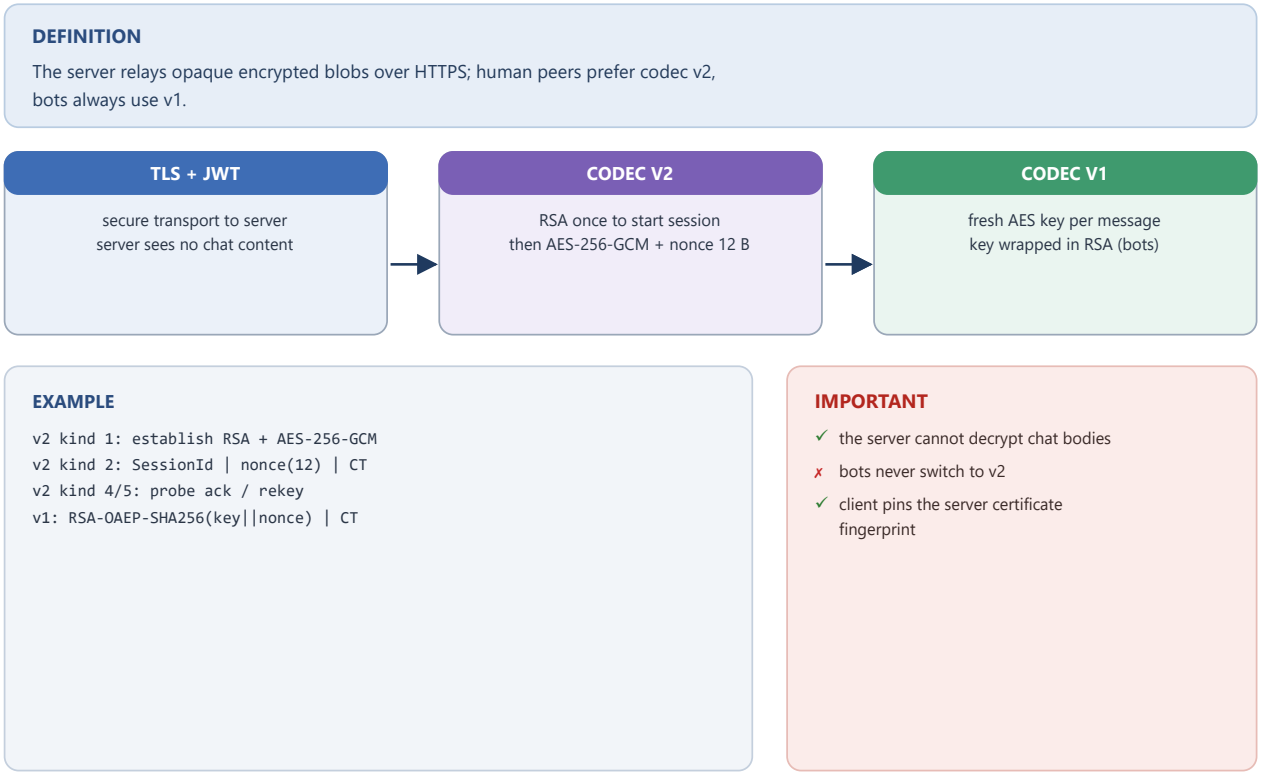
The kind byte values, expanded:
- 0x01 Text — a plain chat text message (UTF-8), what a person types.
- 0x02 Image — an inline picture: MIME type (like "image/png") plus the image bytes.
- 0x03 File — a small file sent inside the chat: file name, MIME type, bytes.
- 0x04 Transfer offer — a JSON description of a big file offered for download outside the chat (which host/port to connect to, a one-time token, size, expiry).
- 0x05 Transfer control — the JSON answer to an offer: accept, reject, or an error.
- 0x06 User info — the sender's profile ("about me" text and avatar image), not shown as a chat bubble.
- 0x07 Vocabulary offer — a list of compression dictionaries the sender supports.
- 0x08 Vocabulary accept — the choice of dictionaries for this chat (or "UNCOMPRESSED").

- 0x09 Compressed text — a text encoded with the chosen dictionary to save traffic (useful on slow radio links).
- 0x0A Session probe — a small "ping" through the encrypted channel to confirm that both ends decrypt correctly.

Not ChatWire frames, but also sent encrypted: 0xAC ACK (message fully received, 17 bytes) and 0xAD NACK (please resend the listed chunk numbers). Very old clients send raw UTF-8 without the S2P1 magic; the receiver still treats it as plain text.

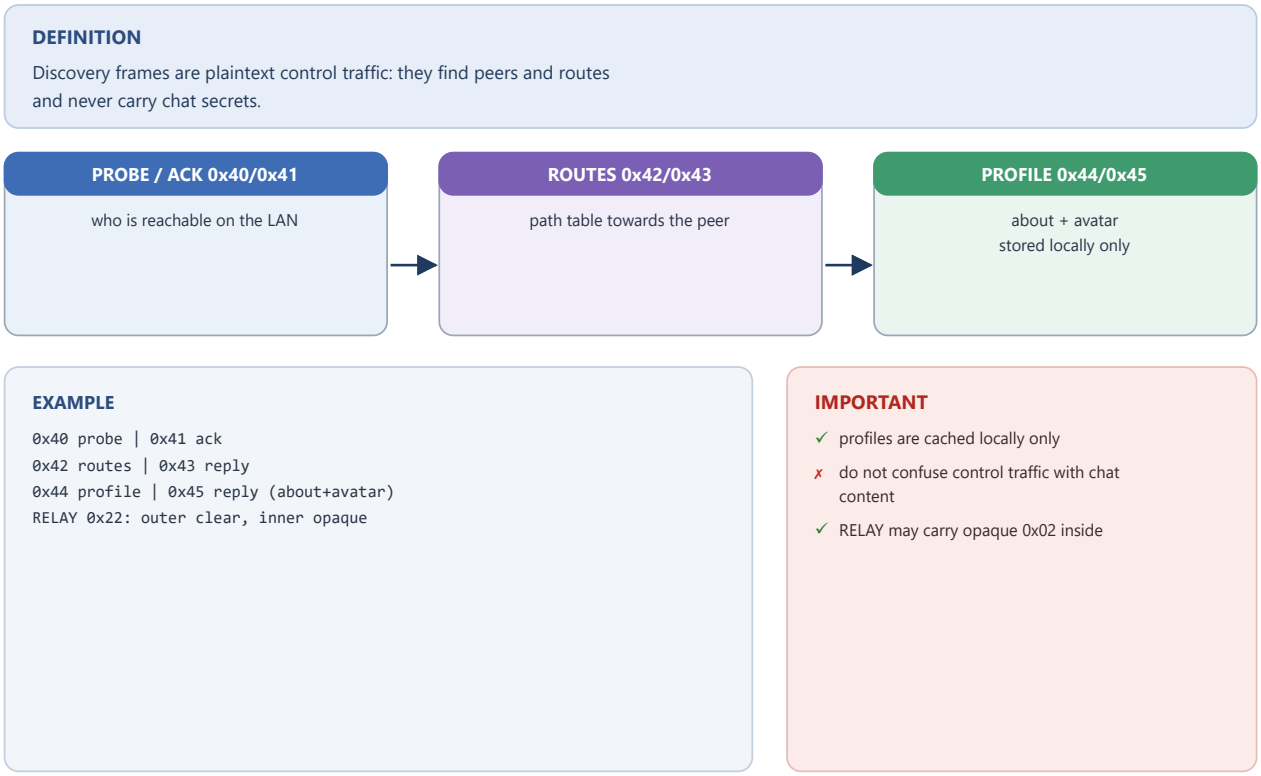
One page per type with byte layouts and unencrypted examples: docs/messages. Crypto pipelines with diagrams: docs/encryption.

5) MESSENGER SERVER AND BOTS



When peers cannot connect directly, they use the Messenger Server over HTTPS/TLS+JWT. The chat body is end-to-end encrypted, so the server stores only an opaque Base64 blob.
- Codec v2 (preferred between people): RSA is used once to start a session; afterwards each message is AES-256-GCM (an authenticated cipher: encryption and integrity check in one algorithm) with a fresh 12-byte random nonce per message.
- Codec v1 (used for bots and as a fallback): every message gets a fresh AES-256-GCM key and nonce, and the key+nonce are wrapped with the recipient's RSA public key (RSA-OAEP-SHA256). Bots always use v1.
In both cases the server cannot decrypt the content.

6) DISCOVERY



Frames 0x40..0x45 on UDP 17890 (probe/ack, route tables, peer profiles) are mostly plaintext control traffic: they help find peers and must not be confused with secret chat content. LAN FIND/FOUND frames locate a peer's address; a RELAY frame can carry an already-encrypted 0x02 payload through intermediate nodes.

FURTHER READING

- docs/encryption — crypto pipelines as vector diagrams (session setup, packet formats, server paths).
- docs/messages — one page per message type with unencrypted byte-level examples.
- protocol.md — detailed frame-level reference (in Russian).